

category: avalonia

tags: [async-await, synchronizationcontext, ui-thread, dispatcher, fire-and-forget, task-run, performance]

severity: high

created: 2026-07-30

updated: 2026-07-30

project-origin: App-Access-V2 (iAccess Desktop v2 Avalonia)

`\_ = SomeAsyncMethod();` (fire-and-forget) KHÔNG tự chạy trên background thread — vẫn đơ UI nếu phía trong có I/O đồng bộ

Tình huống gặp phải

Sửa luồng loading: tách việc tải "controller memory" (gọi API phân trang, có thể nhiều trang nếu nhiều bản ghi) ra khỏi splash Loading (trước đây chặn Main mở), chuyển sang gọi kiểu fire-and-forget ngay sau khi Main đã mở (`_ = LoadControllerMemoryThenSignalReadyAsync();` gọi từ `InitDataAsync`, bản thân được `await` từ `MainViewModel.InitializeAsync()` — chạy trên UI thread vì được gọi từ `Window.Opened` event handler).

Triệu chứng / Lỗi

User báo "ứng dụng rất đơ" sau khi sửa. Không có exception, không crash — chỉ là UI giật/không phản hồi trong lúc dữ liệu đang tải nền. Trước khi sửa (khi việc tải này còn nằm ở splash Loading) không ai để ý vì màn hình Loading vốn là modal chờ, user không mong đợi tương tác được.

Nguyên nhân gốc rễ (Root Cause)

`_ = AsyncMethod();` (fire-and-forget) chỉ có nghĩa là **caller không await kết quả** — nó KHÔNG có nghĩa là code bên trong `AsyncMethod` chạy trên thread khác. Vì `AsyncMethod` được gọi từ UI thread (có `SynchronizationContext` của Avalonia), MỌI continuation sau mỗi `await` bên trong nó (trừ khi dùng `ConfigureAwait(false)` hoặc bọc `Task.Run`) đều được marshal NGƯỢC LẠI UI thread để chạy tiếp.

Cụ thể trong case này: `GetAccessControllerMemories()` là vòng lặp tuần tự theo trang

```
(for (i = 1; i < totalPage; i++) { await GeneralJsonAPIAsync(...);  
SystemUtils.logger.SaveSystemLog(...); })
```

— `SaveSystemLog` ghi SQLite **đồng bộ** (blocking I/O). Vì continuation sau mỗi `await` HTTP resume trên UI thread, đoạn ghi SQLite đồng bộ đó cũng chạy ngay trên UI thread, cho MỌI trang — nếu có nhiều trang (nhiều bản ghi), UI thread bị chặn liên tục trong suốt thời gian tải → app "đơ".

Giải pháp

Bọc TOÀN BỘ phần async có I/O/tính toán nặng trong `Task.Run(...)` — code bên trong chạy trên ThreadPool, KHÔNG có `SynchronizationContext` nên các `await` bên trong không tự marshal về UI thread nữa. Chỉ khi cần cập nhật property bind UI (ObservableCollection, INotifyPropertyChanged) mới `Dispatcher.UIThread.Post(...)` để quay lại UI thread — đúng 1 câu lệnh gọn, không kéo cả block nặng theo.

```
private async Task LoadControllerMemoryThenSignalReadyAsync()
{
    Dispatcher.UIThread.Post(() => IsMemoryLoading = true);
    try
    {
        await Task.Run(async () =>
        {
            // Toàn bộ HTTP + SQLite (đồng bộ) nằm trong đây — chạy trên ThreadPool.
            bool ok = await LoadingWorks.LoadAccessControllerMemoryBackgroundAsync(
                detail => Dispatcher.UIThread.Post(() => MemoryLoadStatusText =
detail));
            // ... xử lý tiếp, chỉ Dispatcher.UIThread.Post khi update property bind UI.
        });
    }
    finally
    {
        Dispatcher.UIThread.Post(() => IsMemoryLoading = false);
    }
}
```

1. Xác định method fire-and-forget nào có vòng lặp/API call nhiều lần + ghi log/DB đồng bộ bên trong.
2. Bọc phần đó trong `Task.Run(async () => { ... })`.
3. Mọi chỗ set property bind UI bên trong → đổi thành `Dispatcher.UIThread.Post(() => prop = value)`.
4. KHÔNG cần bọc `Task.Run` cho method chỉ có 1-2 lần await ngắn, không lặp và không I/O đồng bộ.

Áp dụng lại (How to reuse)

- Khi thấy code Avalonia gọi `_ = SomeAsyncMethod()` (fire-and-forget) từ UI thread (event handler, `Window.Opened`, button click...) MÀ bên trong có vòng lặp gọi API/DB nhiều lần → nghi ngay khả năng đơ UI, kiểm tra có `ConfigureAwait(false)/Task.Run` bọc ngoài không.
- Trước đây việc "chạy chậm nhưng không ai để ý" thường vì nó nằm trong 1 splash/loading screen (user không mong tương tác được lúc đó) — khi dời logic đó sang chạy SAU khi UI chính đã mở (để UX tốt hơn, không chặn user), PHẢI audit lại xem code có giả định "đang chạy trong màn hình chờ nên đơ cũng không sao" hay không.
- `SystemUtils.logger.SaveSystemLog(...)` (LogToSQLite) là ghi SQLite ĐỒNG BỘ — bất kỳ vòng lặp gọi nó nhiều lần liên tiếp trên UI thread đều đáng nghi.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `Task.Run` không tự động đưa TẤT CẢ continuation ra khỏi UI thread nếu code bên trong dùng lại `Dispatcher.UIThread.InvokeAsync` (không phải `.Post`) và AWAIT nó — `InvokeAsync` awaited sẽ chờ UI thread rồi resume ngay trên UI thread. Dùng `.Post` (fire-and-forget, không await) khi chỉ cần set property, không cần đợi UI thread xử lý xong mới tiếp tục.
- ⚠ Đừng bọc `Task.Run` bừa cho MỌI async method — chỉ cần khi có I/O đồng bộ hoặc CPU nặng lặp lại nhiều lần; bọc thừa gây tốn thread pool + phức tạp code không cần thiết.
- ⚠ Việc gọi `AppState.SignalMemoryReady()` / đọc list tĩnh (`AppState.AccessControllerMemories`) từ `ThreadPool` thread bên trong `Task.Run` là AN TOÀN ở đây vì không đụng UI-bound object — nhưng nếu sau này thêm logic đọc/sửa `ObservableCollection` bind UI bên trong `Task.Run`, PHẢI `Dispatcher.UIThread.Post` bọc riêng đoạn đó (không sửa trực tiếp từ background thread).

Tham chiếu

- Pattern tương tự đã có trong cùng codebase: `MainViewModel.OnEventSendToServerAsync` dùng `await Dispatcher.UIThread.InvokeAsync(() => { ... })` để cập nhật `RealtimeEvents` — nhưng đó KHÔNG bọc `Task.Run` phần đọc DB trước nó (nợ kỹ thuật tương tự, chưa audit trong lần sửa này).
- Lesson liên quan: `avalonia-config-window-blocks-ui-thread-sql-timeout.md` (cùng chủ đề UI thread bị block bởi I/O đồng bộ, khác nguyên nhân cụ thể).
- Project liên quan: App-Access-V2 — `MainViewModel.LoadControllerMemoryThenSignalReadyAsync`.